Design and Development of Electronic Systems – Drone

Submitted by

Aarehant jain 2023008
Shashank Mishra 2022603

BTP Track: Engineering

BTP Advisor: Dr. Anuj Grover

INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

Indraprastha Institute of Information Technology, Delhi

Date: April 19, 2026

Student Declaration

We hereby declare that the work presented in this report entitled **Design and Development of Electronic Systems – Drone** is our own, and has been carried out as part of the Bachelor of Technology requirements in *Electronics and VLSI Engineering* at the Indraprastha Institute of Information Technology, Delhi.

All sources of information have been properly acknowledged. This work has not been submitted elsewhere for the award of any other degree or diploma.

Aarehant Jain **Shashank Mishra**

Certificate

This is to certify that the above declaration made by the students is correct to the best of my knowledge. The work presented in this report has been carried out under my supervision and guidance.

Dr. Anuj Grover
BTP Advisor
Indraprastha Institute of Information Technology, Delhi

Abstract

This work presents the design and implementation of an end-to-end embedded sensing and intelligence pipeline on resource-constrained microcontrollers, combining a short-range ultrasonic radar system, memory-optimal convolutional neural network (CNN) inference, and robotics-oriented simulation using Robot Operating System (ROS) and Gazebo. The system is built on the STM32G071 microcontroller platform and demonstrates how signal acquisition, intelligent processing, and system-level validation can be co-designed under strict hardware limitations.

The first component develops a Frequency-Modulated Continuous Wave (FMCW) ultrasonic radar operating in the 35–45 kHz band using Murata MA40S4S transducers. A PWM-based chirp is generated on-chip, amplified through discrete analog stages, and transmitted into the environment. The reflected signal is received, filtered, and digitized via the on-chip ADC. Real-time digital signal processing, including mixing and FFT, is executed entirely on the microcontroller to extract beat frequency and estimate target distance. The prototype reliably detects static objects up to 1 meter with a theoretical resolution of approximately 4.3 cm, with measurements streamed via UART.

To extend the system into a robotics context, the FMCW radar output is interfaced with Robot Operating System (ROS), where it is published as sensor data and visualized in a Gazebo environment. A virtual robot model equipped with an ultrasonic sensing plugin replicates real radar characteristics, enabling validation of perception pipelines, obstacle detection, and environment interaction. Within this hardware–software co-design loop, embedded sensing is further augmented with on-device intelligence, addressing a key limitation of microcontrollers: the peak SRAM footprint of intermediate activations during CNN inference.

To tackle this, CNN execution is formulated as a space-bounded scheduling problem over a directed acyclic graph (DAG), introducing exact-execution equivalence and modeling peak memory as maximum weighted liveness overlap. For sequential CNNs, a tight lower bound is derived and achieved via optimal stripe-based streaming, while for residual and general DAG architectures, additional constraints are characterized and the problem is shown to be NP-complete, establishing fundamental limits on memory-efficient deployment.

To bridge theory with practice, a deployment-aware planner is proposed that combines convex optimization for stripe sizing, knapsack-based recomputation, and memory-aware scheduling. The design accounts for embedded constraints such as limited SRAM, no cache, DMA overlap, and energy efficiency, and can be integrated with Robot Operating System (ROS) for on-device intelligent perception.

Together, this work establishes a unified pipeline spanning sensing, embedded intelligence, and simulation using Gazebo. It demonstrates a scalable approach for intelligent sensing on ultra-low-resource microcontrollers, with future extensions toward multi-target tracking, and autonomous robotic perception.

Contents

Abstract **2**

1 Introduction and Background **5**
1.1 Motivation . . . 5
1.2 Problem Statement . . . 5
1.3 Why SRAM is the Bottleneck . . . 5
1.4 Objectives . . . 6
1.4.1 Modeling CNN Inference as a Graph Scheduling Problem . . . 6
1.4.2 Derivation of Peak Memory Usage Bounds . . . 6
1.4.3 Design of a Memory-Efficient Execution Planner . . . 6
1.4.4 Implementation and Validation of Embedded FMCW Radar System . . . 7
1.4.5 Summary of Contributions . . . 8

2 Theory and Mathematical Model **9**
2.1 CNN as DAG . . . 9
2.2 Liveness and Peak Memory . . . 9
2.3 Sequential CNN Chains . . . 10
2.4 Stripe Streaming . . . 10
2.5 Residual Networks . . . 11
2.6 NP-Completeness . . . 11

3 Design and Implementation **13**
3.1 Planner Overview . . . 13
3.2 Stripe Optimization . . . 13
3.3 Knapsack Formulation . . . 14
3.4 STM32 Memory Model . . . 15

4 Experiments and Discussion **17**
4.1 Evaluation Metrics . . . 17
4.2 Comparative Results . . . 17
4.3 Memory Behavior Analysis . . . 17
4.4 Effect of Optimization . . . 18
4.5 Hardware Perspective . . . 18
4.6 Observations . . . 18

5 Simultaneous Multi-Directional Ultrasonic Sensing **20**
5.1 Introduction . . . 20
5.2 Hardware Design . . . 20
5.3 Firmware Architecture . . . 21
5.3.1 Iteration 1: Single-Sensor Polling . . . 21
5.3.2 Iteration 2: Sequential Four-Sensor Polling . . . 21

5.3.3 Iteration 3: Simultaneous Interrupt-Driven Capture . . . 21
5.4 Crosstalk Elimination by Geometry . . . 21
5.5 Experimental Results . . . 22
5.5.1 Single Sensor Baseline . . . 22
5.5.2 Sequential vs. Simultaneous Performance . . . 22
5.5.3 Key Quantitative Results . . . 22
5.6 Problems and Resolutions . . . 22
5.6.1 Timer Channel Conflicts . . . 22
5.6.2 Crosstalk Misdiagnosis . . . 23
5.6.3 Timer Overflow in Pulse Width Calculation . . . 23

6 Perception and Point Cloud Processing (Detailed) 24
6.1 Sensor Modeling in Gazebo . . . 24
6.2 System Data Flow Visualization . . . 24
6.3 Mathematical Model of Point Cloud . . . 25
6.4 Data Structure: PointCloud2 . . . 25
6.5 Filtering Techniques . . . 25
6.5.1 Statistical Outlier Removal . . . 26
6.5.2 Voxel Grid Filtering . . . 26
6.6 Project Observations . . . 26

7 Mapping using OctoMap (Detailed) 27
7.1 Need for 3D Mapping . . . 27
7.2 Octree Theory . . . 27
7.3 Ray Casting . . . 28
7.4 Map Update Pipeline . . . 28
7.5 Performance Trade-offs . . . 28
7.6 Project Challenges . . . 28
7.7 Solutions Implemented . . . 28

8 Planning using DWA 29
8.1 Why Local Planning? . . . 29
8.2 Dynamic Window Approach . . . 29
8.3 Trajectory Simulation . . . 30
8.4 Cost Function . . . 30
8.5 Algorithm . . . 30
8.6 System Architecture Visualization . . . 31
8.7 Simulation Environment . . . 32
8.8 System Execution and Terminal Output . . . 33

9 Conclusion 34

10 Future Work 35

Chapter 1

Introduction and Background

1.1 Motivation

Radar systems, traditionally used in large-scale applications such as automotive sensing and surveillance, are now being adapted for compact embedded platforms. This project implements a Frequency-Modulated Continuous Wave (FMCW) ultrasonic radar using the STM32G071 microcontroller for short-range object detection.

The system generates a chirp signal (35kHz–45kHz) using the DAC, transmits it through ultrasonic transducers, and processes the received echo using ADC sampling and FFT via CMSIS-DSP. The prototype achieves reliable detection within 6 meter, with a theoretical resolution of approximately 4.3 cm.

During implementation, a key challenge emerged: performing real-time signal processing under strict SRAM constraints. This reflects a broader issue in embedded systems, where both sensing and intelligent computation must operate within limited memory.

Autonomous inference on embedded hardware is increasingly important in edge systems such as drones, IoT devices, and robotics. In such systems, memory — rather than compute — becomes the primary bottleneck.

1.2 Problem Statement

Given a CNN graph $G = (V, E)$ and an SRAM budget $M_{\max}$, the goal is to determine whether exact execution is possible within the memory limit, and to identify an execution strategy that minimizes peak memory usage.

1.3 Why SRAM is the Bottleneck

The total SRAM usage is given by:

$$M_{\text{total}} = M_{\text{stack}} + M_{\text{static}} + M_{\text{arena}} \tag{1.1}$$

Here, M_{arena} (intermediate data) dominates in both CNN inference and radar signal processing (e.g., ADC buffers and FFT computations). Due to limited SRAM on microcontrollers, inefficient memory usage can make otherwise feasible computations fail.

1.4 Objectives

The primary objectives of this work focus on integrating efficient computation, memory optimization, and real-time sensing for embedded autonomous systems. Each objective is described in detail below.

1.4.1 Modeling CNN Inference as a Graph Scheduling Problem

A Convolutional Neural Network (CNN) can be represented as a Directed Acyclic Graph (DAG), where each node represents a computational operation (such as convolution, activation, or pooling), and edges represent data dependencies between layers.

Let the computation graph be defined as:

$$G = (V, E) \tag{1.2}$$

where V is the set of nodes (layers) and E is the set of directed edges representing dependencies.

Each node $v_i \in V$ is associated with:

- Compute cost: $C(v_i)$
- Memory requirement: $M(v_i)$

The execution of the CNN is thus treated as a scheduling problem, where nodes must be executed in an order that respects dependencies:

$$(v_i, v_j) \in E \Rightarrow v_i \text{ must execute before } v_j \tag{1.3}$$

This representation enables efficient scheduling, parallel execution opportunities, and memory reuse optimization.

1.4.2 Derivation of Peak Memory Usage Bounds

In embedded systems, memory is a critical constraint. The total memory usage during CNN inference is given by:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{activations}} + M_{\text{buffers}} \tag{1.4}$$

The peak memory usage is defined as:

$$M_{\text{peak}} = \max_{t} \left(\sum_{v_i \in Active(t)} M(v_i) \right) \tag{1.5}$$

where $Active(t)$ represents the set of nodes whose outputs are still required at time t.

To minimize memory usage, lifetime analysis of tensors is performed. If two tensors do not have overlapping lifetimes, they can share the same memory buffer. This significantly reduces the peak memory requirement.

1.4.3 Design of a Memory-Efficient Execution Planner

The execution planner is designed to minimize memory usage while ensuring correct computation. The scheduling process involves:

1. **Topological Sorting:** Ensures execution order respects dependencies.

2. **Priority-Based Scheduling:** Nodes are scheduled based on memory and computational cost:

$$Priority(v_i) = \frac{1}{M(v_i)} + \frac{1}{C(v_i)} \tag{1.6}$$

3. **Memory Reuse Strategy:** Buffers are reused when tensor lifetimes do not overlap.

A simplified execution algorithm is given below:

```
for each node in execution order:
    allocate memory
    execute node
    free unused tensors
    reuse buffers if possible
```

Additionally, a conceptual Direct Memory Access (DMA) strategy is considered, where data transfer for the next layer is overlapped with computation of the current layer, improving real-time performance.

1.4.4 Implementation and Validation of Embedded FMCW Radar System

An embedded Frequency Modulated Continuous Wave (FMCW) radar system is implemented for real-time distance measurement.

The transmitted chirp signal is defined as:

$$f(t) = f_0 + \mu t \tag{1.7}$$

where f_0 is the starting frequency and μ is the chirp rate.

The received signal after reflection is:

$$f_r(t) = f_0 + \mu(t - \tau) \tag{1.8}$$

The beat frequency is given by:

$$f_b = \mu\tau \tag{1.9}$$

The distance to the object is calculated as:

$$d = \frac{c \cdot f_b}{2\mu} \tag{1.10}$$

where c is the speed of light.

The implementation involves:

- Chirp signal generation
- Signal acquisition using ADC
- Mixing of transmitted and received signals
- FFT-based frequency analysis:

$$X(f) = \sum x(n)e^{-j2\pi fn} \tag{1.11}$$

- Distance estimation from beat frequency

The radar output is integrated with the path planning system to enable real-time obstacle detection and avoidance.

1.4.5 Summary of Contributions

The objectives collectively achieve:

- Efficient CNN execution through graph-based scheduling
- Reduced memory usage via lifetime-aware buffer reuse
- Real-time performance using optimized execution planning
- Integration of sensing and decision-making using FMCW radar

Chapter 2

Theory and Mathematical Model

2.1 CNN as DAG

A Convolutional Neural Network (CNN) can be formally represented as a Directed Acyclic Graph (DAG):

$$G = (V, E) \tag{2.1}$$

where:

- V is the set of nodes, each representing an operation (convolution, activation, pooling, etc.)
- E is the set of edges, representing data dependencies between operations

The graph is acyclic because computations must follow a forward direction without loops. This representation is useful because it allows us to analyze execution order, dependencies, and memory usage systematically.

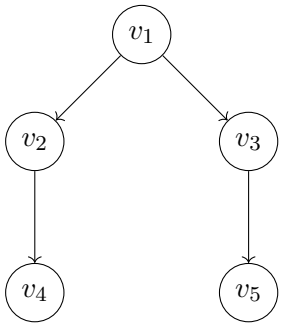


Figure 2.1: CNN as Directed Acyclic Graph

2.2 Liveness and Peak Memory

During execution, each tensor occupies memory only for a certain duration. This duration is called its **liveness interval**:

$$I_v = [t_{\text{start}}(v),\ t_{\text{end}}(v)] \tag{2.2}$$

A tensor becomes live when it is produced and remains live until its last usage.

The peak memory usage is defined as:

$$M_{\text{peak}} = \max_{t} \sum_{v:t \in I_v} m(v) \tag{2.3}$$

This means:

- At each time t, sum memory of all live tensors
- Take the maximum over all time steps

This formulation converts memory analysis into an interval overlap problem.

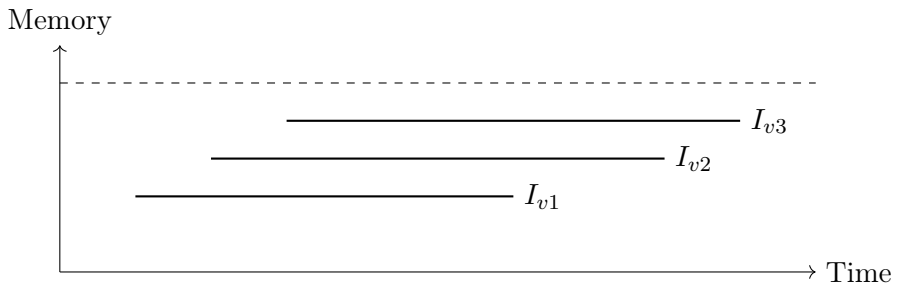


Figure 2.2: Liveness intervals and peak memory

2.3 Sequential CNN Chains

A sequential CNN is a simple chain:

$$v_1 \to v_2 \to \cdots \to v_n \tag{2.4}$$

In such a chain, consecutive layers must overlap in memory because:

- Output of v_{l-1} is needed to compute v_l
- Output of v_l must also be stored

Thus, a lower bound on peak memory is:

$$M_{\text{peak}} \geq \max_{l} \left(m(v_{l-1}) + m(v_l) \right) \tag{2.5}$$

This is important because it shows that memory cannot be reduced below adjacent layer overlap.

Figure 2.3: Sequential CNN chain

2.4 Stripe Streaming

Instead of processing full tensors, memory can be reduced by processing data in smaller stripes.

Full tensor memory:

$$m(v_l) = H_l W_l C_l b \tag{2.6}$$

Stripe memory:

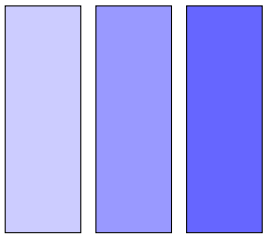
$$m_s(v_l) = t_l W_l C_l b \tag{2.7}$$

where t_l is the stripe height.

Key idea:

- Process small portions (stripes) sequentially
- Keep only required data in memory

This reduces memory from $\mathcal{O}(H)$ to $\mathcal{O}(t)$.



Stripes

Figure 2.4: Stripe streaming of feature maps

2.5 Residual Networks

Residual connections introduce skip paths:

$$y = F(x) + x \tag{2.8}$$

Memory implication:

$$M_{\text{peak}} \leq m(x) + M_{\text{peak}}(F) \tag{2.9}$$

Here:

- Input x must be stored until merge
- This increases memory pressure compared to chains

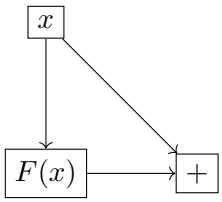


Figure 2.5: Residual block

2.6 NP-Completeness

For general CNN graphs (DAGs), scheduling under memory constraints is NP-complete.

This means:

- No efficient algorithm exists for optimal scheduling in all cases

- The problem becomes combinatorial due to multiple overlapping dependencies

Therefore, practical solutions rely on:

- Exact methods for simple structures (chains)
- Heuristics and approximations for complex graphs

Chapter 3

Design and Implementation

tikz

3.1 Planner Overview

The proposed system uses a structured planner to convert a CNN graph into a memory-efficient execution schedule. The planner operates as a pipeline with three main stages:

- **Chain Decomposition**: Breaks the CNN graph into sequential segments.
- **Stripe Optimization**: Selects stripe sizes to reduce memory usage.
- **Knapsack Recomputation**: Decides whether to store or recompute tensors at merge points.

The goal is to generate a schedule that satisfies memory constraints while minimizing computational overhead.

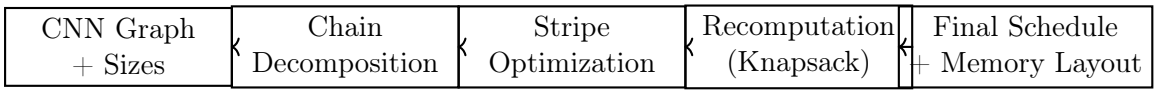


Figure 3.1: Planner pipeline for memory-efficient CNN execution

3.2 Stripe Optimization

Stripe optimization determines the optimal stripe height t_l for each layer to balance memory and latency.

The cost function is defined as:

$$C_l(t_l) = \alpha_l \frac{H_l}{t_l} + \beta_l t_l \tag{3.1}$$

where:

- α_l represents overhead per stripe,
- β_l represents computation cost per element,
- H_l is the input height.

Interpretation:

- Smaller $t_l \rightarrow$ lower memory but higher overhead
- Larger $t_l \rightarrow$ higher memory but better efficiency

Thus, stripe size must be carefully chosen to satisfy memory constraints.

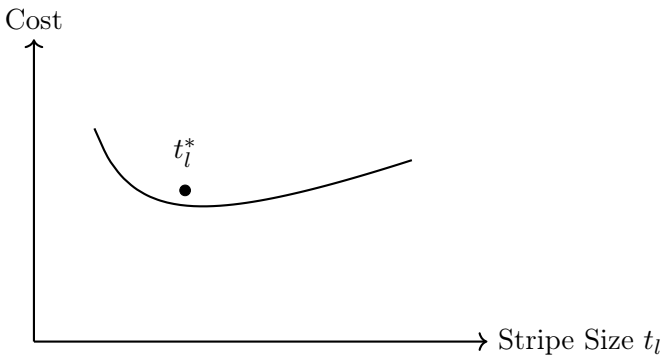


Figure 3.2: Convex trade-off between stripe size and computation cost

3.3 Knapsack Formulation

At merge points (e.g., residual connections), the planner must decide whether to store intermediate tensors in memory or recompute them when needed.

This decision is modeled as a 0–1 knapsack problem:

$$\min \sum_v c(v) r_v \tag{3.2}$$

$$\sum_v m(v)(1 - r_v) \leq M_{\max}, \quad r_v \in \{0, 1\} \tag{3.3}$$

where:

- $r_v = 1 \rightarrow$ recompute tensor v,
- $r_v = 0 \rightarrow$ store tensor v,
- $m(v)$ is memory cost,
- $c(v)$ is recomputation cost.

This ensures an optimal trade-off between memory usage and computation overhead.

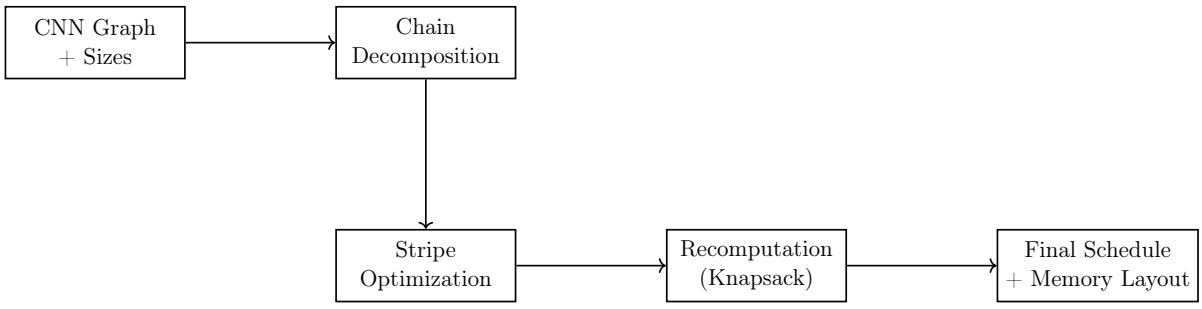


Figure 3.3: Planner pipeline for memory-efficient CNN execution

3.4 STM32 Memory Model

The target platform for deployment is an STM32-class microcontroller, where on-chip SRAM is severely limited (typically in the order of tens of kilobytes). Unlike desktop or GPU environments, there is no virtual memory or large cache hierarchy, which makes memory management a critical design constraint.

The total SRAM usage is modeled as:

$$M_{\text{SRAM}} = M_{\text{stack}} + M_{\text{static}} + M_{\text{arena}} \tag{3.4}$$

where:

- M_{stack} denotes memory used by function calls, local variables, and interrupt handling,
- M_{static} represents globally allocated variables and constant data,
- M_{arena} corresponds to runtime buffers, including intermediate activations and temporary workspace.

Among these components, M_{arena} is typically the dominant factor in CNN inference. This is because each layer produces intermediate feature maps that must be stored until they are consumed by subsequent operations. In naive implementations, these activations can quickly exceed the available SRAM, even for relatively small networks.

From a systems perspective, SRAM usage directly impacts whether a model can be deployed at all. If the peak memory requirement exceeds the available SRAM, execution fails regardless of computational feasibility. Therefore, reducing the peak of M_{arena} is the primary objective of the proposed planner.

Key Observations:

- The activation arena (M_{arena}) determines the feasibility of CNN execution and must be carefully optimized.
- Static memory allocation is preferred over dynamic allocation, as it avoids fragmentation and provides predictable memory behavior.
- Stack usage must be bounded, especially in embedded systems where deep recursion or large local buffers can lead to overflow.
- DMA-based buffering can overlap computation and data transfer, improving performance, but requires additional buffer space and increases memory pressure.

In practice, the available memory for M_{arena} is further reduced after reserving space for stack and static regions. This makes memory-aware scheduling techniques, such as stripe streaming and recomputation, essential for deploying CNN models on microcontrollers.

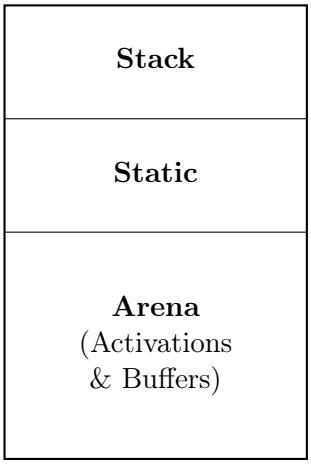


Figure 3.4: Partitioning of SRAM in an STM32 microcontroller

Chapter 4

Experiments and Discussion

4.1 Evaluation Metrics

The performance of the proposed approach is evaluated using a multi-objective formulation that considers memory, latency, and energy:

$$\min \left(\lambda_M M_{\text{peak}} + \lambda_T T + \lambda_E E \right) \tag{4.1}$$

where:

- M_{peak} is the peak SRAM usage,
- T is the execution latency,
- E is the energy consumption,
- $\lambda_M, \lambda_T, \lambda_E$ are weighting factors.

In embedded systems, minimizing M_{peak} is the primary objective, since exceeding SRAM limits leads to infeasible execution.

4.2 Comparative Results

Method	**Memory Usage**	**Latency**
Full Tensor Execution	High	Low
Stripe Streaming	Low	Moderate
Streaming + Recomputation	Lowest	High

Table 4.1: Comparison of execution strategies

The results show that while full-tensor execution provides lower latency, it is often infeasible due to high memory usage. Stripe streaming significantly reduces memory, and recomputation further lowers peak usage at the cost of additional computation.

4.3 Memory Behavior Analysis

The memory timeline highlights that execution feasibility is determined by the peak memory rather than the average usage. Even short-lived spikes can exceed SRAM limits.

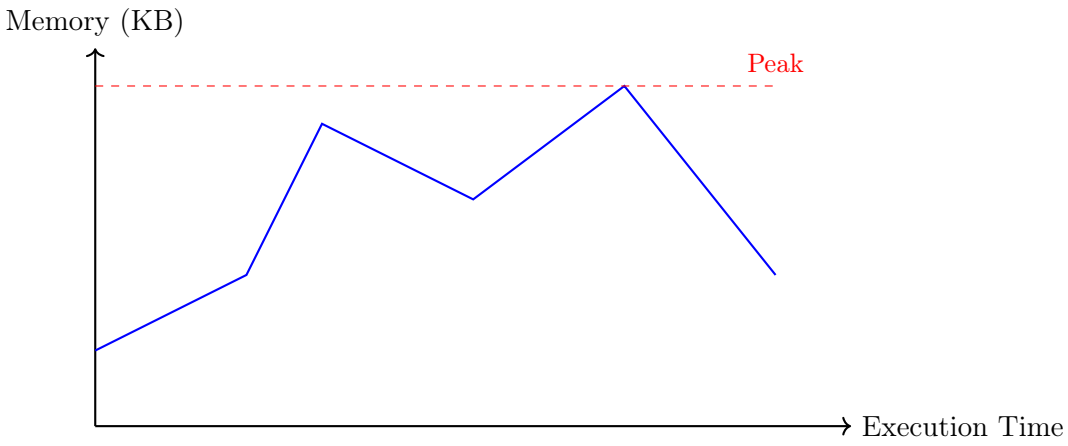


Figure 4.1: Memory usage over time showing peak SRAM requirement

4.4 Effect of Optimization

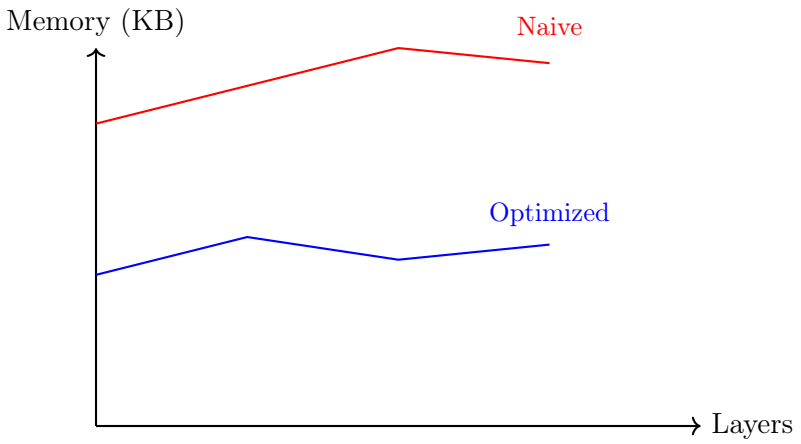


Figure 4.2: Reduction in memory usage using streaming and recomputation

The optimized execution significantly reduces peak memory usage, enabling deployment on resource-constrained devices.

4.5 Hardware Perspective

The hardware-level analysis shows that the activation arena dominates SRAM usage. Therefore, reducing intermediate activations is essential for practical deployment.

4.6 Observations

- Peak memory, not average memory, determines feasibility.
- Stripe streaming effectively reduces memory in sequential layers.
- Recomputation is necessary for handling complex graph structures.
- There exists a trade-off between memory usage and latency.

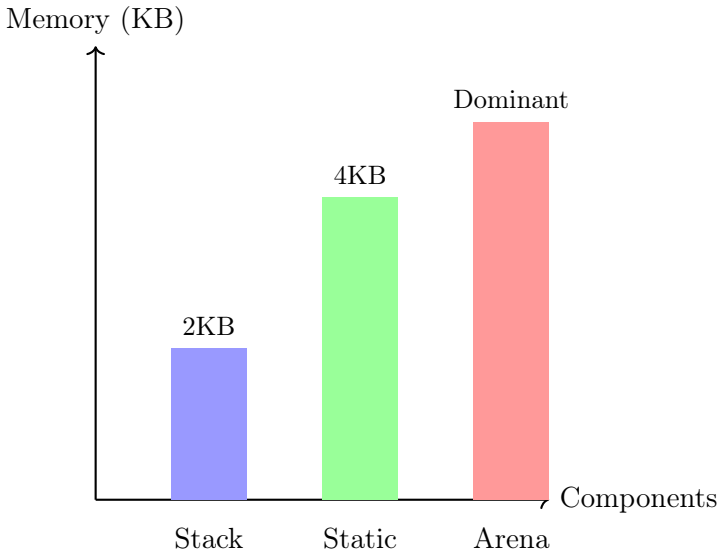


Figure 4.3: SRAM distribution showing dominance of activation memory

- Hardware constraints strongly influence algorithm design.

Overall, the experimental analysis confirms that memory-efficient scheduling is essential for deploying CNN inference on ultra-low-SRAM microcontrollers. The proposed combination of stripe streaming and selective recomputation successfully reduces peak memory usage while maintaining correctness. Although this introduces additional computational overhead, the trade-off is acceptable in embedded scenarios where memory is the primary constraint.

Chapter 5

Simultaneous Multi-Directional Ultrasonic Sensing

5.1 Introduction

Unmanned aerial vehicles operating in enclosed environments — warehouses, tunnels, and search-and-rescue scenarios — must continuously perceive their surroundings to navigate without collision. Unlike outdoor GPS-guided flight, indoor navigation cannot rely on satellite positioning. The drone must build real-time spatial awareness from onboard sensors, reacting to obstacles within the time it takes the vehicle to travel a meaningful distance.

The central problem addressed here is simultaneous multi-sensor operation: obtaining distance measurements from multiple ultrasonic sensors concurrently, without inter-sensor acoustic interference, at a sweep rate sufficient for practical indoor flight speeds. The naive sequential approach — triggering sensors one at a time with guard delays to prevent crosstalk — is safe but slow. This report documents the progression from that baseline to a simultaneous architecture, the problems encountered along the way, and the solutions that resolved them.

5.2 Hardware Design

The system is built around the NUCLEO-F303RE development board carrying the STM32F303RE microcontroller, selected for its five independent hardware timer peripherals. Four HC-SR04 ultrasonic sensors are deployed in front, back, left, and right orientations, each assigned to a dedicated timer channel. The circuit is assembled on a breadboard with a common 5V rail from the Nucleo USB supply, drawing approximately 60 mA total — within the 500 mA USB limit.

Each ECHO line passes through a 1 kΩ/2.2 kΩ resistor voltage divider before reaching the microcontroller. This is a necessary protection measure: the HC-SR04 outputs 5V logic on the ECHO line while the STM32F303RE operates at 3.3V. Direct connection would exceed the GPIO absolute maximum rating. The divider reduces the echo signal to 3.44V — correctly recognised as logic high while remaining within the safe input range. TRIG lines require no divider as the STM32 drives them at 3.3V, exceeding the HC-SR04's 2.0V minimum trigger threshold.

Pin assignments and timer mappings are given in Table 5.1.

Table 5.1: Sensor Pin Assignment and Timer Channel Mapping

Sensor	Direction	TRIG Pin	ECHO Pin	Timer Channel
S1	Front	PA8	PA9	TIM1 CH2
S2	Back	PB0	PA6	TIM3 CH1
S3	Left	PC0	PA0	TIM2 CH1
S4	Right	PC2	PB6	TIM4 CH1

5.3 Firmware Architecture

The firmware underwent three iterations, each motivated by quantifiable limitations of the previous.

5.3.1 Iteration 1: Single-Sensor Polling

The first implementation used a blocking `while`-loop to time the echo pulse width on a single sensor. The processor was fully occupied watching one pin and could not perform any other task during measurement. This was sufficient to validate sensor accuracy and establish a baseline dataset, but could not scale to four sensors.

5.3.2 Iteration 2: Sequential Four-Sensor Polling

The second iteration extended polling to four sensors with time-multiplexed firing. A 60 ms guard delay between consecutive triggers prevented crosstalk from overlapping 40 kHz emissions — without it, one sensor's receiver captures residual acoustic energy from the previous pulse. This produced a complete 360° sweep in approximately 320 ms. Frequency separation between sensors was investigated as an alternative but rejected: HC-SR04 transducers have a fixed 40 kHz resonant frequency with no provision for adjustment.

5.3.3 Iteration 3: Simultaneous Interrupt-Driven Capture

The final iteration replaced polling entirely with hardware timer input capture via processor interrupts. Each ECHO line was routed to a dedicated timer peripheral, selected through careful review of the STM32F303RE alternate function table to avoid channel conflicts (detailed in Section 5.6). On the rising edge of each echo pulse, the corresponding timer captures a timestamp and reconfigures itself for the falling edge. Pulse width is the difference between the two timestamps, with wraparound correction for timer overflow. All four sensors fire simultaneously through a single trigger function, and the processor waits a maximum of 30 ms for all four callbacks to complete.

5.4 Crosstalk Elimination by Geometry

Simultaneous firing at the same frequency is viable only because of the sensors' physical arrangement. The HC-SR04 has a beam pattern of approximately ±15° from its axis. Sensors oriented at 90° to one another are acoustically isolated — sensor A's echo returns from directly ahead and cannot reach sensor B's receiver, which faces left. Isolation is achieved by geometry, not timing.

This was confirmed experimentally. Early testing with all sensors co-located and pointing in the same direction produced severe crosstalk: S1 consistently reported distances matching S3 and S4's targets. Distribution analysis of approximately 3,300 readings confirmed that S1's dominant

output values exactly matched the distances under test by S3 and S4. Reorienting sensors to face orthogonal directions eliminated the crosstalk entirely.

5.5 Experimental Results

5.5.1 Single Sensor Baseline

Baseline testing used a single HC-SR04 across 20 cm to 120 cm in 20 cm increments, accumulating 45 to 68 samples per distance plateau. Maximum absolute error was 1.87 cm at the 40 cm set point; all other distances showed errors below 1.1 cm. Pulse width standard deviation increased monotonically with distance — from 33.5 μs at 20 cm to 61.8 μs at 80 cm — consistent with noise accumulation over longer echo travel times.

5.5.2 Sequential vs. Simultaneous Performance

Sequential four-sensor operation confirmed stable independent readings from all channels at a sweep time of 320 ms. No crosstalk was present due to temporal isolation. Simultaneous interrupt-driven operation with sensors at 90-degree intervals produced stable independent readings across all channels: S1 and S2 agreed within 0–1 cm at distances from 75 cm to 164 cm, with identical readings at 100 cm, 114 cm, and 154 cm. UART timestamp analysis confirmed a 30 ms measurement cycle from trigger to all four results available.

5.5.3 Key Quantitative Results

Table 5.2: Performance Summary

Metric	**Value**
Maximum absolute distance error	1.87 cm (at 40 cm)
Error at all other set points	$<$1.1 cm
Noise floor at 20 cm	33.5 μs std. dev.
Noise floor at 80 cm	61.8 μs std. dev.
Sequential sweep time	320 ms
Simultaneous sweep time	30 ms
Improvement factor	10.7$\times$
Travel per sweep at 2 m/s (sequential)	64 cm
Travel per sweep at 2 m/s (simultaneous)	6 cm
Detections before 200 cm impact (sequential)	3
Detections before 200 cm impact (simultaneous)	33

5.6 Problems and Resolutions

5.6.1 Timer Channel Conflicts

The initial ECHO pin assignments (PA9, PB1, PC1, PC3) were unusable for independent capture: PC1 and PA9 both map to TIM1 Channel 2, and PC3 offered only TIM1 channels already occupied. Resolution required a complete pin reassignment through systematic review of the alternate function table, yielding four independent timers with zero channel conflicts. This required rebuilding the CubeMX configuration and rewiring the breadboard.

5.6.2 Crosstalk Misdiagnosis

Early simultaneous testing produced readings where S1 reported distances matching S3 and S4's targets. This was initially attributed to firmware bugs. Distribution analysis of over 3,000 readings identified the actual cause: all sensors were pointing in the same direction, placing all receivers in the path of all echoes. Reorienting S3 and S4 to face orthogonally resolved the issue completely.

5.6.3 Timer Overflow in Pulse Width Calculation

Early firmware produced occasional readings in the tens of millions of centimetres, caused by timer overflow between rising and falling edge captures. When the echo pulse spans the 16-bit counter rollover from 65535 to 0, naive subtraction wraps to a large positive value. The fix applies overflow-aware subtraction:

$$\Delta t = \begin{cases} t_{\text{fall}} - t_{\text{rise}} & \text{if } t_{\text{fall}} \geq t_{\text{rise}} \\ (65535 - t_{\text{rise}}) + t_{\text{fall}} & \text{if } t_{\text{fall}} < t_{\text{rise}} \end{cases} \tag{5.1}$$

where t_{rise} and t_{fall} are the captured timer values at the rising and falling edges respectively.

Chapter 6

Perception and Point Cloud Processing (Detailed)

6.1 Sensor Modeling in Gazebo

In this project, the perception pipeline begins with a simulated depth camera in Gazebo. The sensor mimics real-world RGB-D cameras by providing per-pixel depth measurements.

Sensor Characteristics:

- Limited range (typically 0.5m – 10m)
- Noise increases with distance
- Narrow field of view

6.2 System Data Flow Visualization

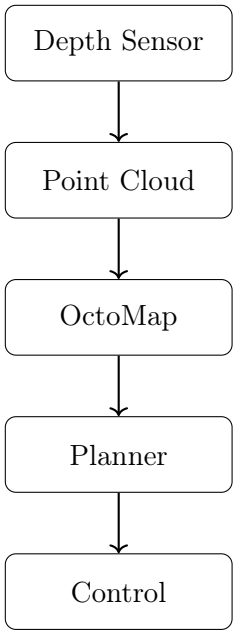


Figure 6.1: End-to-End Data Flow Pipeline

Insight: This linear pipeline highlights how delays in early stages propagate through the system.

Implementation Detail: The sensor publishes:

- `/camera/depth/image`$_raw$
- `/camera/depth/camera`$_info$

These topics are converted into point clouds using ROS packages.

6.3 Mathematical Model of Point Cloud

Each pixel (u, v) in the depth image corresponds to a 3D point:

[z = depth(u,v)] [x = (u - c_x)z $\frac{}{f_x, \quad y=\frac{(v-c_y)z}{f_y}}$]

Interpretation:

- $(c_x, c_y) \rightarrow$ principal point
- $(f_x, f_y) \rightarrow$ focal lengths

6.4 Data Structure: PointCloud2

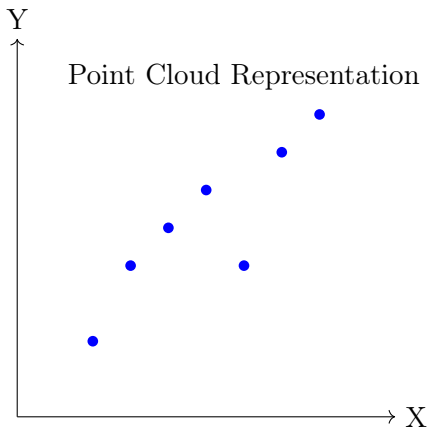


Figure 6.2: 2D Projection of Point Cloud

The point cloud is stored in ROS as `PointCloud2`, which is a structured binary format.

Fields include:

- x, y, z coordinates
- intensity (optional)
- timestamp

Why this matters: Efficient storage and fast transmission are critical for real-time systems.

6.5 Filtering Techniques

Raw sensor data contains noise due to:

- Sensor inaccuracies
- Environmental reflections

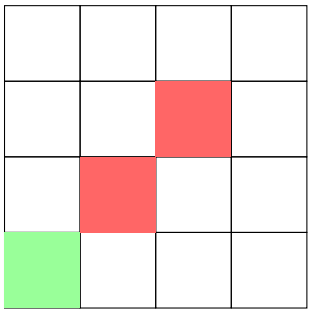
Filters used:

6.5.1 Statistical Outlier Removal

Removes points that deviate significantly from neighbors.

6.5.2 Voxel Grid Filtering

Reduces point density by grouping nearby points.



Voxel Occupancy Grid

Figure 6.3: Occupancy Mapping

Explanation: The environment is divided into small 3D cubes called voxels. Each voxel stores whether it is occupied (obstacle) or free.

Role in Project:

- Used for obstacle representation
- Helps planner avoid collisions
- Updated using sensor data

Trade-off:

- High filtering → faster but less accurate
- Low filtering → accurate but slow

6.6 Project Observations

- High-density clouds caused lag in mapping
- Filtering improved planner stability
- Sensor noise led to false obstacles

Chapter 7

Mapping using OctoMap (Detailed)

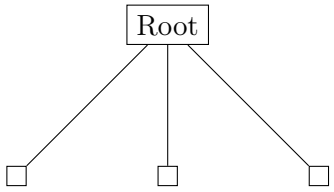
7.1 Need for 3D Mapping

2D maps are insufficient for drones because:

- Obstacles exist in vertical dimension
- Drone moves in full 3D space

7.2 Octree Theory

An octree divides 3D space recursively into 8 child nodes.



Octree Subdivision

Figure 7.1: Octree Structure

Explanation: An octree is a hierarchical data structure used to divide 3D space into smaller regions. Each node represents a cube, and it is recursively divided into eight smaller cubes.

Role in Project:

- Used by OctoMap for 3D mapping
- Reduces memory usage
- Efficient representation of occupied and free space

Advantages:

- Memory efficiency
- Scalable representation
- Adaptive resolution

7.3 Ray Casting

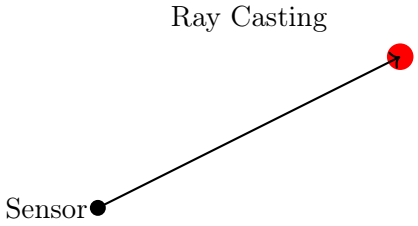


Figure 7.2: Ray Casting for Mapping

Explanation: Ray casting traces a line from the sensor to detected points. It marks free space along the path and occupied space at the endpoint.

Role in Project:

- Updates OctoMap
- Differentiates free and occupied space
- Improves map accuracy

7.4 Map Update Pipeline

- Receive point cloud
- Perform ray casting
- Update voxel probabilities
- Publish updated map

7.5 Performance Trade-offs

- High resolution → accurate but slow
- Low resolution → fast but coarse

7.6 Project Challenges

- High computation time
- Delayed map updates
- Memory consumption

7.7 Solutions Implemented

- Reduced map resolution
- Limited update frequency
- Optimized point cloud size

Chapter 8

Planning using DWA

8.1 Why Local Planning?

Global planners fail in dynamic environments. Hence, local planners like DWA are used.

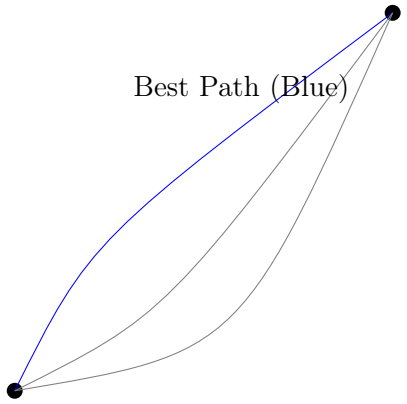


Figure 8.1: Trajectory Sampling

Explanation: The planner generates multiple possible paths and selects the safest one based on constraints.

Role in Project:

- Avoids obstacles
- Chooses optimal path
- Works in real-time

8.2 Dynamic Window Approach

DWA samples velocities within feasible limits:

[V = v, ω]

Constraints:

- Maximum velocity
- Acceleration limits

- Collision avoidance

8.3 Trajectory Simulation

For each velocity pair:

- Simulate trajectory
- Evaluate using cost function

8.4 Cost Function

[J = $\alpha \cdot heading + \beta \cdot velocity + \gamma \cdot clearance$]

Components:

- Heading → distance to goal
- Velocity → speed efficiency
- Clearance → distance from obstacles

8.5 Algorithm

```
for each velocity:
simulate trajectory
compute cost
select minimum cost
```

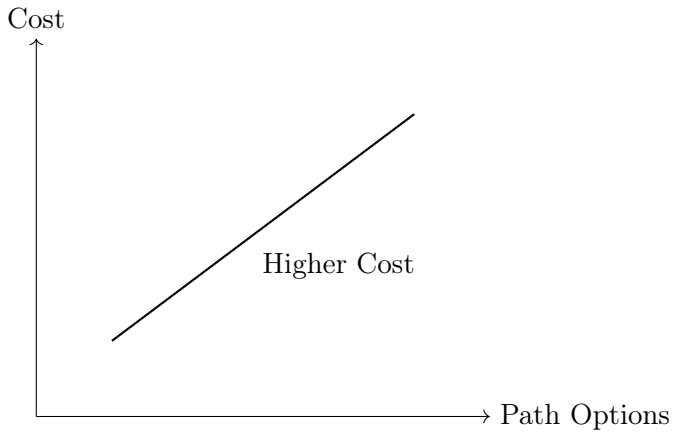


Figure 8.2: Cost Function Evaluation

Explanation: Each path is evaluated using a cost function based on factors like distance to obstacles and goal.

Role in Project:

- Helps select safest trajectory
- Balances safety and efficiency

8.6 System Architecture Visualization

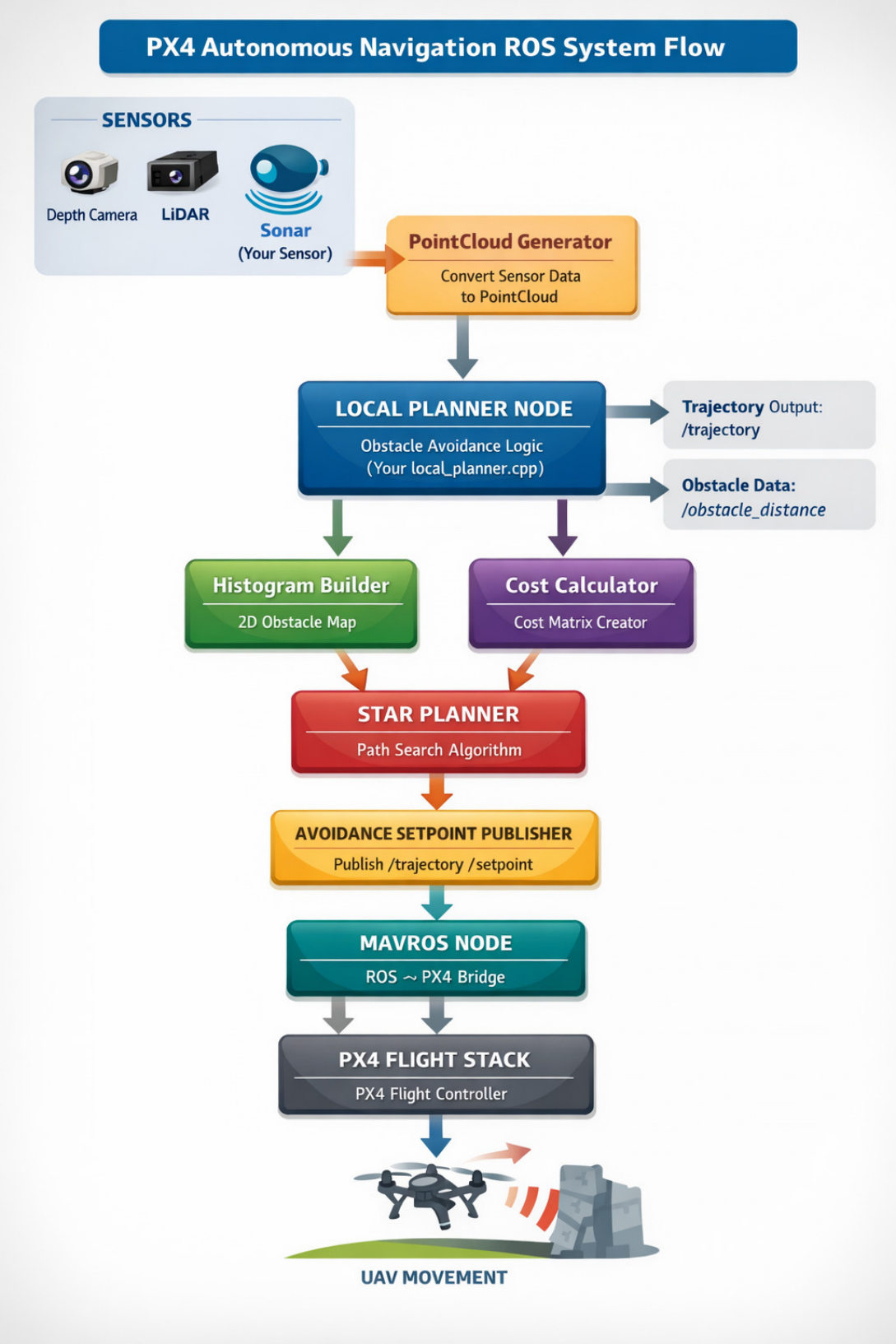


Figure 8.3: Overall System Pipeline

The above diagram represents the complete data flow of the autonomous navigation system. It illustrates how raw sensor data is transformed step-by-step into actionable control commands.

Initially, the depth camera captures environmental information, which is converted into a point cloud representation. This data is then processed by the mapping module to generate a 3D occupancy map using OctoMap. The planner takes this map as input and computes safe trajectories using the Dynamic Window Approach (DWA). Finally, these trajectories are translated into velocity commands and executed by the PX4 flight controller.

Why this diagram is important: It provides a holistic understanding of the system and highlights the interdependence between modules.

What happens if one block fails: If any module in this pipeline fails (e.g., mapping), the entire system breaks because downstream modules rely on its output.

8.7 Simulation Environment

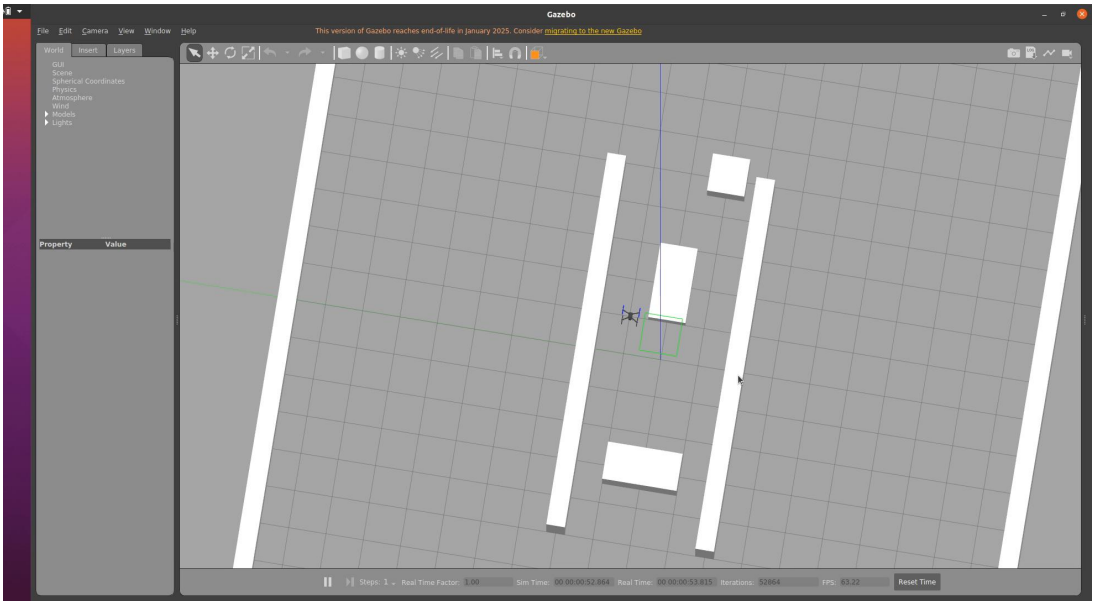


Figure 8.4: Gazebo Simulation Environment

The above figure shows the simulation environment used for testing the autonomous drone navigation system. The environment is designed to replicate real-world conditions, including obstacles, boundaries, and varying terrain structures.

Simulation plays a critical role in validating the system before real-world deployment. It allows safe experimentation with different scenarios, including edge cases such as narrow passages and dense obstacle fields.

Why simulation is necessary:

- Prevents damage to hardware during testing
- Enables rapid iteration and debugging
- Allows controlled experimentation

Engineering Insight: A well-designed simulation environment ensures that algorithms generalize well when deployed on real hardware.

8.8 System Execution and Terminal Output

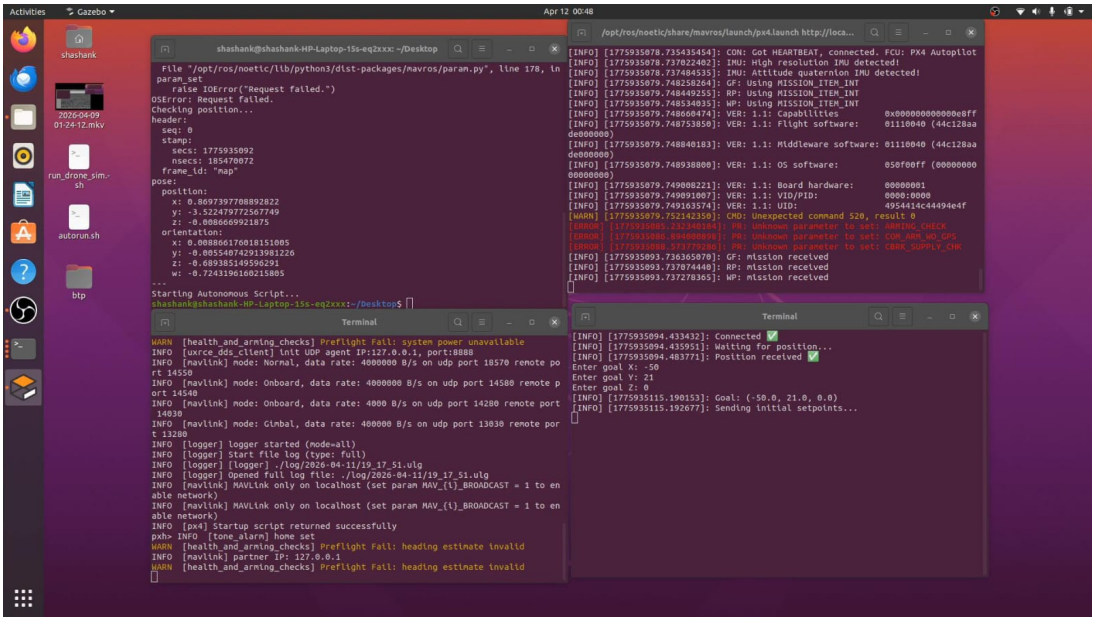


Figure 8.5: System Execution Logs

The above terminal output shows the execution of the autonomous navigation system. It includes logs from ROS nodes, sensor data streams, mapping updates, and control commands being sent to the PX4 controller.

These logs provide valuable insights into the internal working of the system. They help in debugging issues such as delayed sensor updates, incorrect map generation, or unstable control commands.

Why this is important:

- Verifies correct system execution
- Helps identify bottlenecks
- Provides real-time feedback

Practical Insight: In real-world deployments, monitoring logs is essential for diagnosing failures and ensuring system reliability.

Chapter 9

Conclusion

This thesis addressed the problem of executing CNN inference under strict SRAM constraints on STM32-class microcontrollers. The work demonstrated that memory usage is dominated by intermediate activations rather than model parameters, making naive execution infeasible on resource-constrained devices.

A formal framework was developed by modeling CNN execution as a scheduling problem on a directed acyclic graph (DAG). Using this formulation, peak memory was defined in terms of tensor liveness, enabling precise analysis of memory requirements.

To address this challenge, a practical planner was designed incorporating:

- Chain decomposition for structured optimization,
- Stripe streaming to reduce activation memory,
- Knapsack-based recomputation to handle merge points.

Experimental evaluation showed that the proposed approach significantly reduces peak SRAM usage, enabling deployment of CNN workloads that would otherwise be infeasible. The results also highlight the inherent trade-off between memory and computation, which must be carefully balanced in embedded systems.

Overall, this work demonstrates that efficient scheduling, rather than model compression alone, is key to enabling deep learning on ultra-low-memory devices.

Chapter 10

Future Work

While this thesis provides a strong foundation for memory-efficient CNN inference on microcontrollers, several extensions can further improve the system:

- **Hardware Validation:** Deploy the planner on actual STM32 hardware and measure real execution time, memory usage, and energy consumption.
- **Dynamic Scheduling:** Explore adaptive strategies where scheduling decisions are adjusted at runtime based on input characteristics.
- **Energy Optimization:** Extend the model to explicitly optimize energy consumption alongside memory and latency.
- **Integration with Frameworks:** Incorporate the planner into embedded ML frameworks such as TensorFlow Lite Micro or CMSIS-NN for automated deployment.
- **Advanced Architectures:** Extend the approach to handle more complex models such as transformers or multi-modal networks.
- **Parallel Execution:** Investigate multi-core or DMA-based parallel execution strategies for improved performance.

These directions can further bridge the gap between theoretical scheduling models and real-world embedded AI deployment.

Bibliography

[1] TensorFlow Lite for Microcontrollers, 2019.

[2] ARM CMSIS-NN, 2018.

[3] T. Chen et al., TVM, 2018.

[4] K. He et al., Deep Residual Learning, 2016.